BFF Pattern: An Example

Let's take a look at a simple Java example implementing the BFF pattern.

Coding Example

Let's say we have an e-commerce application with a web frontend and a mobile app.

1. Product Service (Main Backend Service):

- Provides product information.

2. Web BFF:

- Tailored for web browsers.
- Provides extensive product details.

3. Mobile BFF:

- Tailored for mobile apps.
- Provides concise product information to save data and improve load times.

1. Product Service (Main Backend)

This is the main backend microservice that deals with products.

```java
@RestController
@RequestMapping("/products")
public class ProductServiceController {

  @Autowired
  private ProductRepository productRepository;

  @GetMapping("/{productId}")
  public Product getProductById(@PathVariable String productId) {
    return productRepository.findById(productId)
                            .orElseThrow(() -> new
ProductNotFoundException(productId));
  }
}
```

In this simple controller, we have a method `getProductById` that fetches a product based on its ID. It interacts with a `ProductRepository` to fetch product data from the database. The `Product` class would contain all the detailed information about a product.

2. Web BFF

The Web BFF is tailored for the web frontend.

```java
@RestController
@RequestMapping("/web/products")
public class WebProductController {

  @Autowired
  private ProductServiceClient productServiceClient;

  @GetMapping("/{productId}")
  public ProductDetails getProductDetails(@PathVariable String productId) {
    Product product = productServiceClient.getProductById(productId);
    return convertToWebProductDetails(product);
  }

  private ProductDetails convertToWebProductDetails(Product product) {
    // Logic to convert product to a detailed view suitable for web.
    // This could include adding more descriptions, images, etc.
  }
}
```

Here, `ProductServiceClient` is a Feign client or any HTTP client that you are using to communicate with the Product Service. The `getProductDetails` method fetches a product and then converts it to a `ProductDetails` object, which is a detailed view tailored for web users.

3. Mobile BFF

The Mobile BFF is tailored for the mobile frontend.

```java
@RestController
@RequestMapping("/mobile/products")
public class MobileProductController {

  @Autowired
  private ProductServiceClient productServiceClient;

  @GetMapping("/{productId}")
  public SimpleProductDetails getProductDetails(@PathVariable String productId) {
    Product product = productServiceClient.getProductById(productId);
    return convertToMobileProductDetails(product);
  }

  private SimpleProductDetails convertToMobileProductDetails(Product product) {
    // Logic to convert product to a concise view suitable for mobile.
    // This could include stripping down descriptions, using smaller images, etc.
  }
}
```

In the Mobile BFF, the `getProductDetails` method fetches the product just like in the Web BFF, but it converts the product to a `SimpleProductDetails` object, which is a more concise view tailored for mobile users.

In Action

So in this setup:

- The main Product Service handles all the heavy lifting related to product data.
- The Web BFF provides a detailed view of the product, suitable for web users.
- The Mobile BFF provides a concise view of the product, suitable for mobile users.

Each frontend gets exactly what it needs, and the backend logic is tailored to provide the best possible user experience.

← Previous
The Architecture Of The BFF Pattern

Next →
Performance Implications

☐ Mark as Completed